

AAA LEARNING SESSION

RAG vs Graph RAG vs Hybrid

Real use cases, trade-offs, and step-by-step implementation

Traditional RAG

Graph RAG

Hybrid

Code walkthrough + Colab notebook + session notes included

What we'll cover today



The RAG premise

Why retrieval-augmented generation exists



Traditional RAG

Pipeline, strengths, and where it breaks



Graph RAG

Knowledge graphs, local vs global search



Hybrid + routing

Best of both, decided per query



Real use cases

Support, healthcare, legal, enterprise

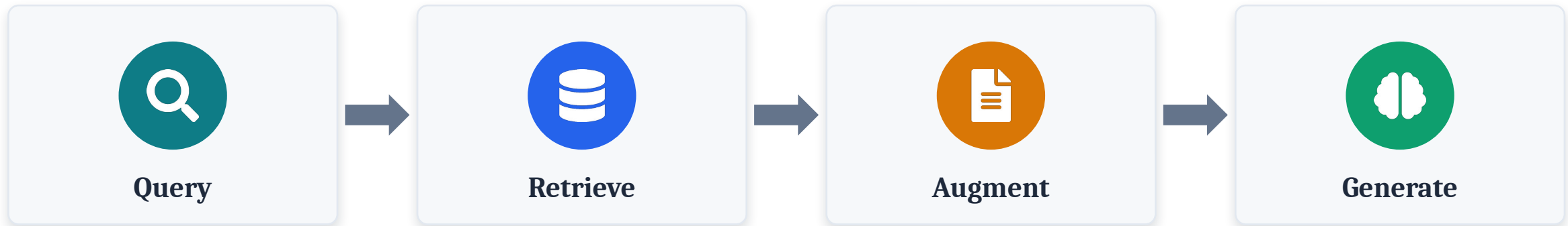


Hands-on build

Step-by-step Colab walkthrough

Why retrieval? The RAG premise

An LLM only knows its training data. RAG injects fresh, private, or domain knowledge into the prompt at query time — so answers are grounded in your data, not just the model's memory.



The retrieval step is the whole game

Generation quality is capped by what you retrieve. How you index and search your knowledge — flat vectors vs. a graph vs. a mix — is exactly what separates the three approaches we cover today.

Traditional RAG — how it works



Offline: Load → Chunk → Embed → Store | At query time: Retrieve → Generate

Mental model

A smart librarian with sticky notes. Each chunk becomes a point in vector space. A query is embedded the same way, and the system returns the nearest passages. It is fast, cheap, and *flat* — every chunk stands alone, with no awareness of how it connects to the others.

Traditional RAG — strengths & best fit



Strengths

- Simple to build and operate
- Fast responses, low latency
- Low infrastructure & token cost
- Frequent data updates are trivial (just re-embed)
- Great for single-document, fact-based questions
- Mature tooling (LangChain, LlamaIndex, FAISS, Chroma)



Best-fit use cases



Customer support & FAQ bots



Document & policy search



Internal knowledge bases



Semantic search over manuals



Chat over a single report/PDF

Where naive RAG breaks down



Global questions

“What are the main themes across all docs?”
Top-k similarity only sees a few chunks, never the whole corpus.



Multi-hop reasoning

Answers needing $A \rightarrow B \rightarrow C$ across documents fail — related facts live in chunks that aren't textually similar.



Scattered context

Entities described in many places get fragmented; the model never assembles the full picture.

The root cause: vectors capture *similarity*, not *relationships*. Graph RAG adds the missing structure — entities and the links between them.

Graph RAG — what it is

Instead of a flat list of chunks, build a knowledge graph: extract entities (nodes) and relationships (edges), then group densely-connected nodes into community summaries.



Entities

People, orgs, concepts → nodes



Relationships

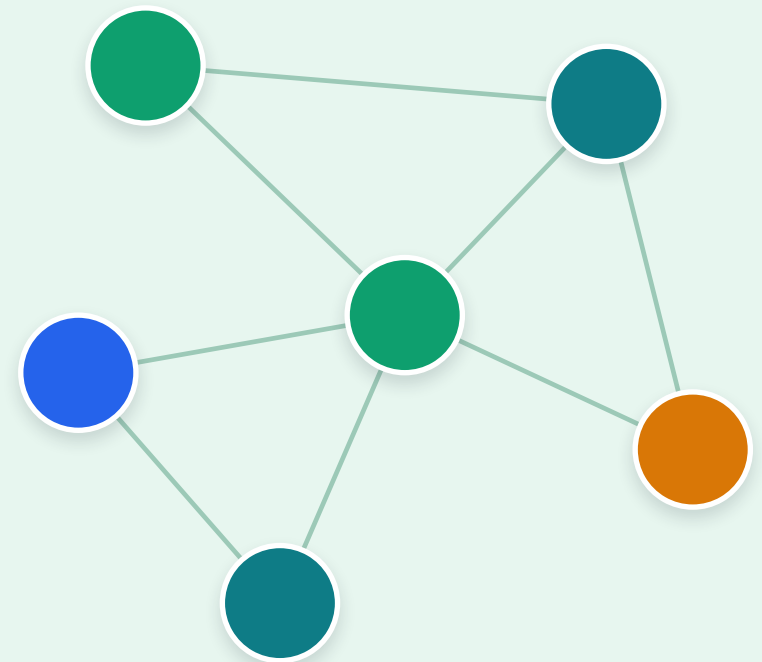
How entities connect → edges



Communities

Clusters + LLM summaries

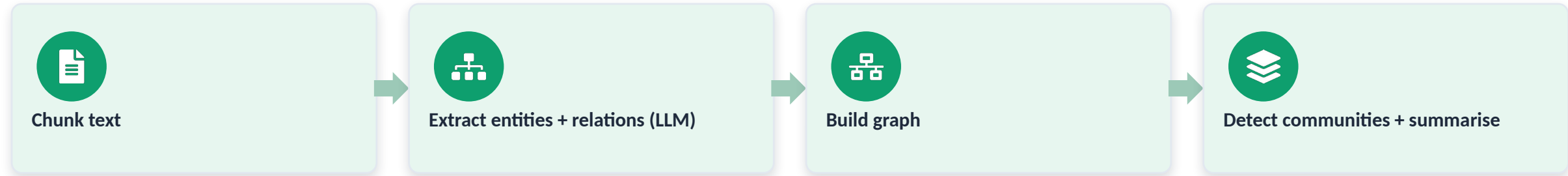
Knowledge graph



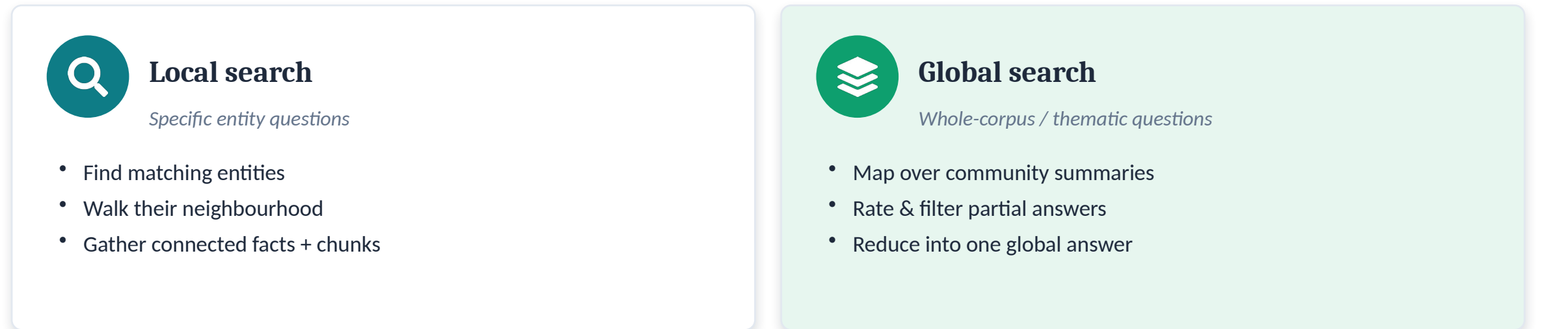
Nodes = entities · edges = relationships

Graph RAG — indexing & query

INDEXING (offline)



QUERY (two search modes)



Graph RAG — strengths & best fit



Strengths

- Connects facts across many documents
- Multi-hop & relationship reasoning
- Answers global, thematic questions
- Higher explainability — traceable paths
- Richer, more comprehensive responses



Best-fit use cases



Healthcare & biomedical Q&A



Legal research & case law



Enterprise knowledge management



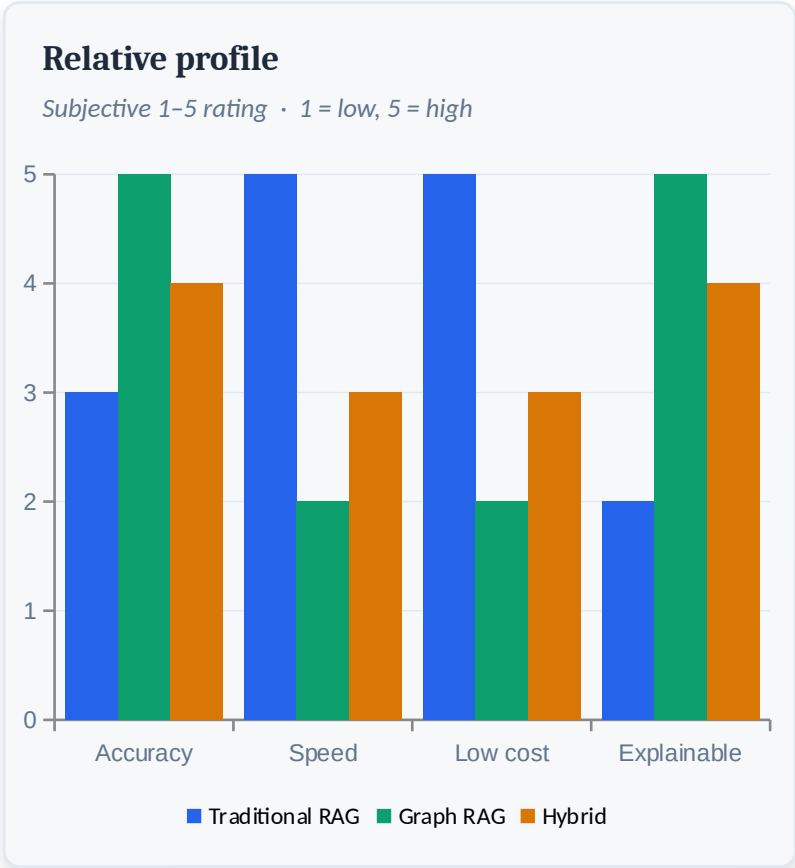
Compliance & audit reasoning



Fraud / investigation graphs

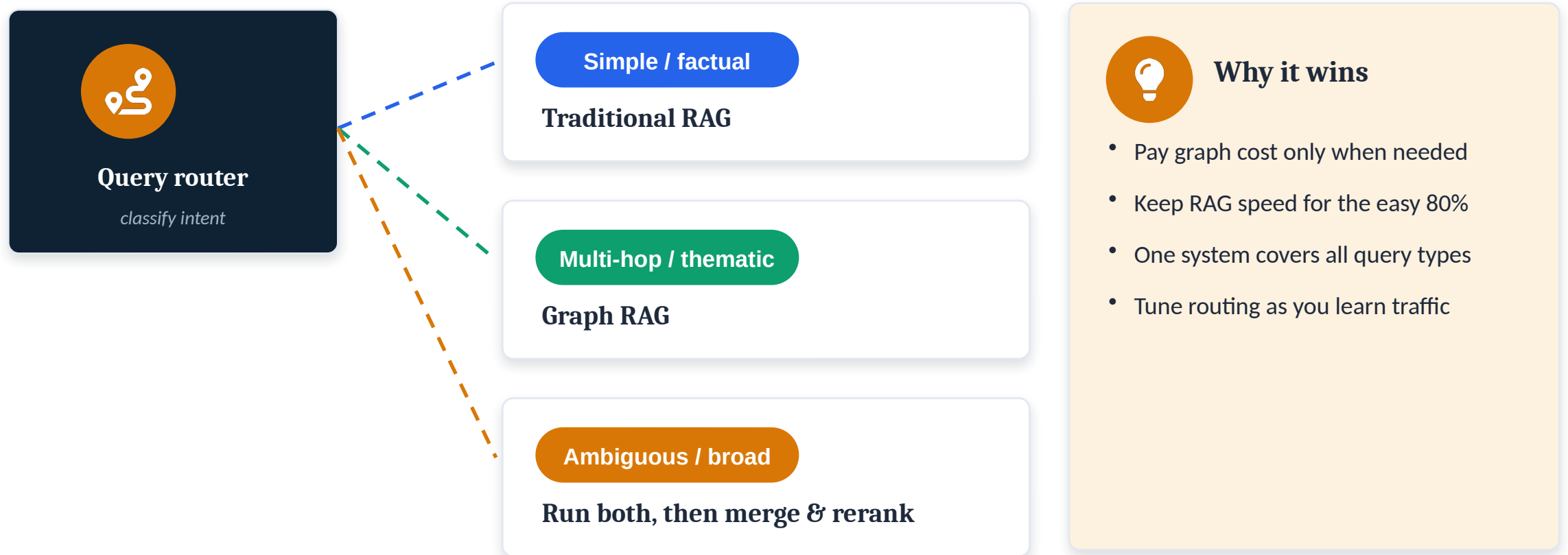
Side-by-side trade-offs

Factor	Traditional RAG	Graph RAG	Hybrid
Best question type	Fact-based, local	Exploratory, multi-hop	Mixed workloads
Accuracy on complex Qs	Medium	Very high	High-Very high
Speed / latency	Fast	Slower	Routed
Explainability	Lower	Higher (paths)	Higher
Update speed	Fast (re-embed)	Slower (re-index)	Mixed
Relative cost	Low	Higher	Medium-High



The hybrid approach — best of both

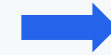
Don't pick one architecture for every query. Put a lightweight router in front that classifies each query and sends it down the cheapest path that can answer it well.



Which one should I use?



Is the question fact-based and answerable from one place?



Use Traditional RAG



Does it span many documents or need multi-hop reasoning?



Use Graph RAG



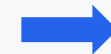
Is traffic mixed, or are you unsure per query?



Use Hybrid + routing



Tight latency / budget and simple data?



Stay with Traditional RAG

Real use cases, side by side



Support assistant

Traditional RAG

- Answers from help-centre articles
- Single-doc, factual queries
- Latency & cost sensitive
- Refresh content daily



Clinical knowledge base

Graph RAG

- Links symptoms, drugs, studies
- Multi-hop medical reasoning
- Needs traceable evidence
- Accuracy over speed



Enterprise “ask-your-docs”

Hybrid

- Mix of lookups & analysis
- Route simple vs. complex
- Graph for cross-team themes
- RAG for quick facts

Implementation — the stack we'll use



LLM + embeddings

OpenAI (gpt-4o-mini) or any provider; sentence-transformers for local



Vector store

FAISS or Chroma — fast similarity search for the RAG path



Graph layer

networkx graph from LLM-extracted entities & relations (concepts behind GraphRAG / LightRAG)



Router

Lightweight LLM classifier picks RAG vs Graph vs both



Orchestration

LangChain to glue loading, chunking, retrieval & generation

What the Colab notebook builds

1

Traditional RAG

Load → chunk → embed → FAISS → retrieve → answer

2

Graph RAG (from scratch)

LLM extracts entities/relations → networkx graph → traverse → answer

3

Hybrid router

Classify the query → route to RAG, Graph, or both

4

Compare & evaluate

Run the same questions through all three; inspect differences

CURATED, CURRENT AS OF JUNE 2026

Top references & high-star repos



microsoft / graphrag

~33.6k★

The canonical Graph RAG system: entity graph + community summaries, local & global search.



HKUDS / LightRAG

~36.5k★

Simple & fast KG-RAG; dual-level retrieval, cheap incremental updates. Great GraphRAG alternative.



gusye1234 / nano-graphrag

hackable

Tiny, readable GraphRAG implementation — ideal for learning the internals.



NirDiamant / RAG_Techniques

~27.8k★

40+ notebook tutorials: simple, adaptive, corrective, agentic & graph RAG.



neo4j / neo4j-graphrag

~5k★

Build KGs from unstructured data (llm-graph-builder); production graph retrievers on Neo4j.



Awesome-GraphRAG (DEEP-PolyU)

list

Curated surveys, papers, benchmarks & open-source GraphRAG projects.

Pitfalls & best practices



Common pitfalls

- Reaching for Graph RAG when RAG suffices
- Underestimating graph indexing cost & time
- Poor chunking — too big or too small
- No evaluation harness (flying blind)
- Stale index after data changes
- Ignoring retrieval quality, blaming the LLM



Best practices

- Start with RAG; add graph for the hard 20%
- Measure: faithfulness, recall, relevancy (Ragas)
- Tune chunk size & overlap to your docs
- Add reranking before generation
- Cache embeddings; re-index incrementally
- Keep a golden Q&A set for regression tests

Takeaways

Retrieval is the architecture

- RAG = fast, cheap, flat — best for local, factual questions.
- Graph RAG = relationships & global reasoning, at higher cost.
- Hybrid = route per query; pay for depth only when you need it.
- Evaluate retrieval first — most quality issues live there.



Open discussion: Built any RAG, GraphRAG, Agent, MCP, CrewAI, LangGraph, ADK or A2A project? Bring a demo or a learning to share.